CAMPUSFLOW

CampusFlow

Student Portal | Project Phase 1

Course: CSCI426 - Advanced Web Programming

Semester: Fall 2026-2027

Student: Akil Sheikh Hussein

Student ID: 12032486

Application type: Responsive frontend React web application

Live production: https://campusflow-csci426.vercel.app/

GitHub: Repository publishing requires account authorization; the source archive includes a local Git history and GitHub publishing scripts.

PROJECT SUMMARY

A student-first academic workspace for managing courses, planning assignments, following deadlines, and monitoring semester progress. Built with React, Vite, client-side routing, responsive CSS and browser storage.

1. Abstract

CampusFlow is a responsive, frontend-only academic management web application developed for CSCI426 Advanced Web Programming. Students can monitor courses, update progress and grades, manage study tasks, track upcoming deadlines and review personal performance. The prototype illustrates modern React component design, dynamic client-side navigation, state-driven interaction, browser persistence and mobile-first user experience.

2. Problem and Project Objectives

Students frequently distribute deadlines, notes and grade calculations across multiple unrelated tools. CampusFlow consolidates common planning activities in a single dashboard that reduces navigation effort and provides actionable academic indicators.

- Provide a consistent, accessible interface on desktop, tablet and mobile.
- Support course creation, course search and dynamic per-course views.
- Manage task details, priority, due date, completion and deletion.
- Display performance summaries based on locally stored course and task data.
- Demonstrate a real-world React SPA with deployment and version control.

3. System Design and Information Flow

Architecture: The application is a single-page React client. React Router renders each URL route, while the main App component owns course, task and theme state. Child components handle feature-specific interactions. React effects persist changes in localStorage; on reload, saved state is loaded from the current browser.

Layer	Responsibility
Presentation	React components and CSS layout: dashboard, cards, forms, responsive navigation
Navigation	React Router; static pages and dynamic /courses/:id route
Logic	React state, event handlers, filtering, grade and task summary calculations
Persistence	Browser localStorage for courses, tasks and theme preferences
Hosting	Vercel production deployment of the Vite build output

Information flow: User action → React handler → state update → React rerender → localStorage synchronization. Reloading reverses the flow by reading saved data back into state.

4. Functional Screens and Routes

Page / URL	Main capabilities
/	Overview dashboard, KPIs, upcoming tasks and course shortcuts
/courses	Search and add courses; inspect progress cards
/courses/:id	Dynamic details, grade and progress sliders, linked assignments
/planner	Create, filter, complete and delete dated study tasks
/performance	Grade bars, course averages and completion metrics
/about	Platform overview and primary use cases
/contact	Validated demo form and a local confirmation message

5. Interface Design and Responsiveness

The interface uses a dark charcoal base, vivid purple accent, clearly differentiated priority badges, consistently shaped content cards and readable typographic hierarchy. CSS media queries collapse desktop grids for smaller screens. Below 800 px, the fixed desktop sidebar becomes a toggleable navigation drawer; narrower phone layouts stack cards and forms in one column. The user can switch to light mode, with the preference stored locally.

6. Implementation Technologies

Technology	Use in CampusFlow
React 19	Reusable components, controlled inputs, hooks and event-driven updates
React Router 7	Route handling for static and dynamic course pages
Vite 6	Development environment and optimized production build
CSS3	Responsive breakpoints, grid and flex layouts, theme variants
Lucide React	Consistent vector icons throughout the interface
localStorage	Persistent, browser-local demonstration data
Git and Vercel	Source history and hosted production build

7. Key Implementation Excerpts

A. Load persisted data safely:

```
function stored(key, fallback) {
  try {
    return JSON.parse(localStorage.getItem(key)) ?? fallback;
  } catch { return fallback; }
}
```

B. Save study tasks when their state changes:

```
useEffect(() => {
  localStorage.setItem('cf_tasks', JSON.stringify(tasks));
}, [tasks]);
```

C. Resolve dynamic course pages:

```
<Route path="/courses/:id"
       element={<CourseDetail courses={courses}
                              tasks={tasks}
                              setCourses={setCourses} />} />
```

D. Toggle task completion without mutating existing state:

```
setTasks(tasks => tasks.map(t =>
  t.id === selectedId ? { ...t, done: !t.done } : t
));
```

8. Build, Deployment and Verification

Confirmed production build: On 10 November 2026, the Vercel deployment dpl_FcSpZr4fGFoRwVgJMjv85gKmaDye successfully completed npm installation and the Vite production build. The Vercel build log reported 1,590 modules transformed and a successful output deployment; project state was READY. The production hostname is https://campusflow-csci426.vercel.app/.

Verified scope: Successful installation, compilation and production deployment from the provided source. Interactive cross-browser and phone viewport tests were not independently executed within this report environment; they should be recorded before course submission.

9. Manual User Acceptance Checklist

☐ Navigate through all seven page types; refresh a deep-linked course URL.

☐ Add a course, search by its name/code and adjust its detail-page sliders.

☐ Add a task, mark it complete, filter results, remove it and refresh to confirm persistence.

☐ Submit invalid and valid values on the demo contact form.

☐ Verify desktop and mobile layouts at 1440px, 768px and 390px.

☐ Capture actual browser screenshots and add them to README.md.

☐ Publish the preserved local Git history to the student-owned GitHub repository and include its final URL.

10. Limitations and Next Phase

This Phase 1 project intentionally has no backend, real authentication, data sharing, payment features or outgoing emails. Data is saved only in each visitor’s browser and is erased when that storage is cleared. Seeded sample records are illustrative. A later phase could add authentication, a hosted database, calendars and teacher-student collaboration.

11. Academic Integrity and Asset Credits

The CampusFlow implementation was prepared as original coursework assistance rather than copied from a prebuilt application template. Interface icons are from Lucide React; display fonts use Google Fonts. The student should review and understand the React components, state updates, route structure and persistent storage before presenting the project.